《计算机组成原理》实验报告

年级、专业、班级		**姓名**	
实验题目	实验四简单五级流水线 CPU		
实验时间	2025 年 5 月 13 日	**实验地点**	DS1410
实验成绩	优秀/良好/中等	**实验性质**	□验证性 ☑设计性 □综合性

教师评价：

□算法/实验过程正确； □源程序/实验内容提交； □程序结构/实验步骤合理;

□实验结果正确； □语法、语义正确； □报告规范;

其他:

评价教师: 钟将

实验目的

(1)掌握流水线 (Pipelined) 处理器的思想。

(2)掌握单周期处理中执行阶段的划分。

(3)了解流水线处理器遇到的冒险。

(4)掌握数据前推、流水线暂停等冒险解决方式。

报告完成时间: 2025 年 5 月 29 日

1 实验内容

阅读实验原理实现以下模块：

(1) Datapath，所有模块均可由实验三复用，需根据不同阶段，修改 mux2 为 mux3(三选一选择器)，以及带有 enable(使能)、clear(清除流水线) 等信号的触发器，
(2) Controller，其中 main decoder 与 alu decoder 可直接复用，另需增加触发器在不同阶段进行信号传递
(3) 指令存储器 inst_mem(Single Port Ram)，数据存储器 data_mem(Single Port Ram)；同实验三一致，无需改动，
(4) 参照实验原理，在单周期基础上加入每个阶段所需要的触发器，重新连接部分信号。实验给出 top 文件，需兼容 top 文件端口设定。
(5) 实验给出仿真程序，最终以仿真输出结果判断是否成功实现要求指令。

2 实验设计

本节详细阐述简单五级流水线 CPU 的设计，涵盖流水线总体结构、关键模块（数据通路、控制器、冒险处理模块）的功能、接口、逻辑控制方法及状态机实现。重点介绍提前判断分支、数据前推、流水线暂停和清空等控制逻辑。

2.1 流水线 CPU 总体设计

五级流水线 CPU 将指令执行分为取指（Instruction Fetch, IF）、译码（Instruction Decode, ID）、执行（Execute, EX）、访存（Memory Request, MEM）和写回（Write Back, WB）五个阶段，通过流水线寄存器分隔，实现指令的并行处理。相比单周期 CPU，流水线提高吞吐量，但需处理数据冒险（如 load-use）和控制冒险（如分支、跳转）。关键机制包括：

- 提前判断分支：在译码阶段比较分支条件，尽早确定分支跳转，减少错误指令的执行。
- 数据前推：将执行、访存或写回阶段的结果直接传递给后续指令，减少数据冒险的暂停。
- 流水线暂停：在 load-use 冒险等场景中暂停取指和译码阶段，插入气泡以等待数据准备。
- 流水线清空：在分支或跳转时清空错误取指的指令，确保流水线正确性。

各阶段功能如下：

- IF：从指令存储器（inst_mem）读取指令，更新 PC（PC+4、分支或跳转地址）。
- ID：解析指令，读取寄存器堆，生成控制信号，提前判断分支条件。
- EX：执行 ALU 运算，计算分支/访存地址，应用数据前推（实际修改后分支地址在 ID 阶段计算）。
- MEM：访问数据存储器（data_mem），执行 load/store 操作。
- WB：将结果（ALU 或内存数据）写回寄存器堆。

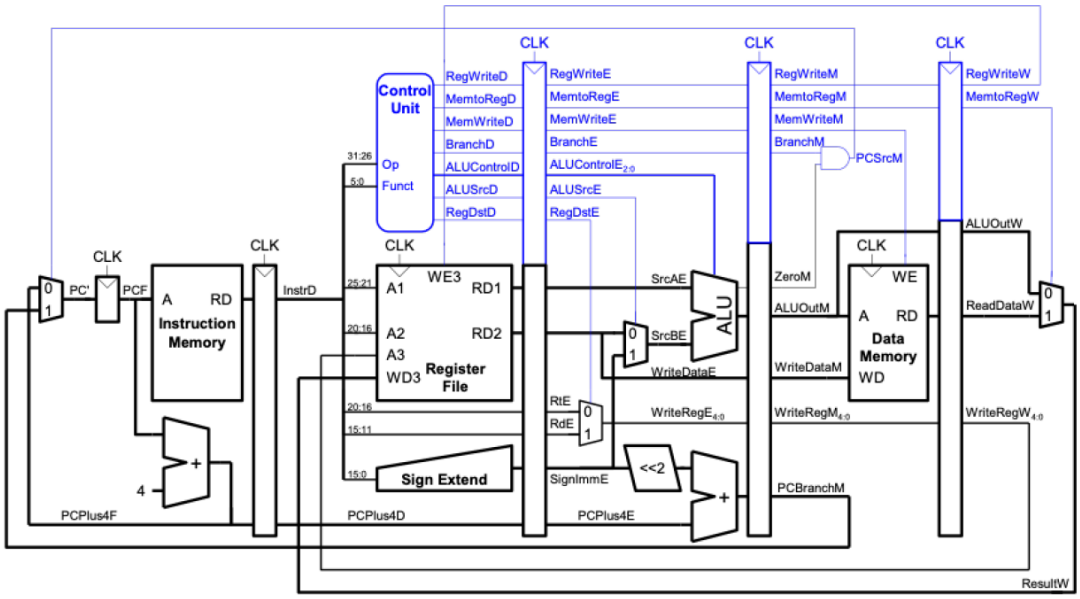


图 1: 加入控制器的流水线示意图

参考“加入控制器的流水线示意图”,展示了数据通路、控制信号和流水线寄存器的整体结构。

2.2 冒险处理模块

2.2.1 功能描述

冒险处理模块检测并解决数据冒险（EX-EX、MEM-EX、load-use）和控制冒险（分支、跳转）。通过数据前推、暂停和清空机制确保指令正确执行。

参考“数据前推示例”，展示了前推路径;“流水线暂停示例”说明 load-use 冒险的暂停逻辑;“提前判断分支示例”突出分支前推。

2.2.2 接口定义

见表 1。

2.2.3 逻辑控制方法

冒险处理模块通过组合逻辑生成控制冒险的输出信号（forwardAE/BE、forwardAD/BD、stallF、stallD、flushE、valid），基于输入信号（寄存器地址和控制信号）实时检测数据和控制冒险。这些信号由 Datapath 使用以实现冒险解决（如 mux 选择、寄存器暂停、清空）。以下按冒险类型分析信号的逻辑控制过程,说明每个信号的触发条件、涉及的阶段和数据、对应的冒险类型及解决方式。

1. 数据冒险：数据冒险发生在指令依赖尚未写回寄存器堆的数据，包括 EX-EX、MEM-EX 和 load-use 冒险。信号通过数据前推或流水线暂停解决。

表 1: 冒险处理模块接口定义

信号名	方向	位宽	功能描述
rsD, rtD	Input	5-bit	译码阶段源寄存器 1, 源寄存器 2 地址。
rsE,rtE	Input	5-bit	执行阶段源寄存器 1, 源寄存器 2 地址。
writeregE	Input	5-bit	执行阶段写回寄存器地址。
writeregM	Input	5-bit	访存阶段写回寄存器地址。
writeregW	Input	5-bit	写回阶段写回寄存器地址。
regwriteE	Input	1-bit	执行阶段寄存器写使能信号。
regwriteM	Input	1-bit	访存阶段寄存器写使能信号。
regwriteW	Input	1-bit	写回阶段寄存器写使能信号。
memtoregE	Input	1-bit	执行阶段内存到寄存器选择信号。
memtoregM	Input	1-bit	访存阶段内存到寄存器选择信号。
branchD	Input	1-bit	译码阶段分支指令标志。
forwardAE	Output	2-bit	ALU 输入 A 前推选择。
forwardBE	Output	2-bit	ALU 输入 B 前推选择(同上)。
forwardAD	Output	2-bit	分支比较寄存器 1 前推选择。
forwardBD	Output	2-bit	分支比较寄存器 2 前推选择(同上)。
stallF	Output	1-bit	取指阶段暂停信号。
stallD	Output	1-bit	译码阶段暂停信号。
flushE	Output	1-bit	执行阶段清空信号。

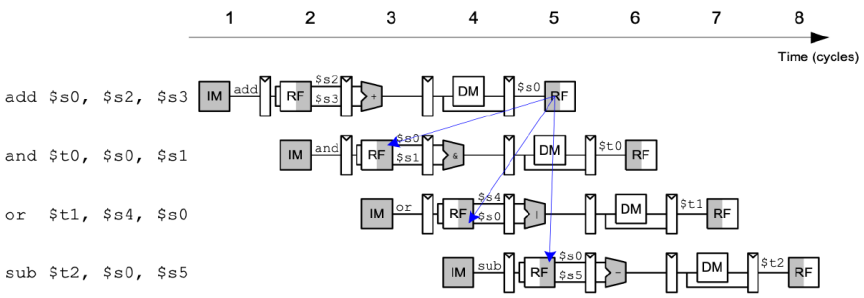


图 2: 数据冒险示例

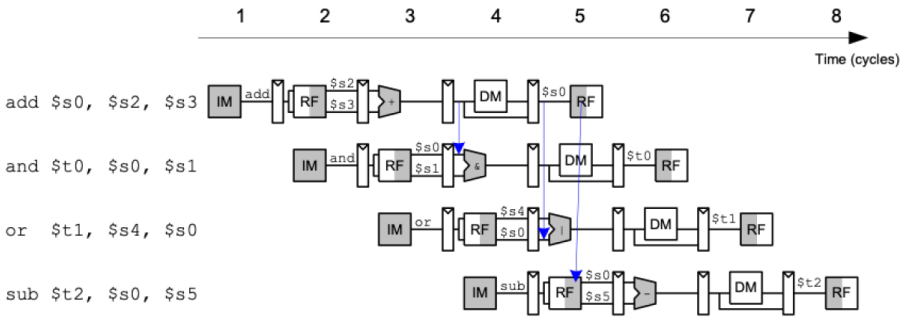


图 3: 数据前推示例

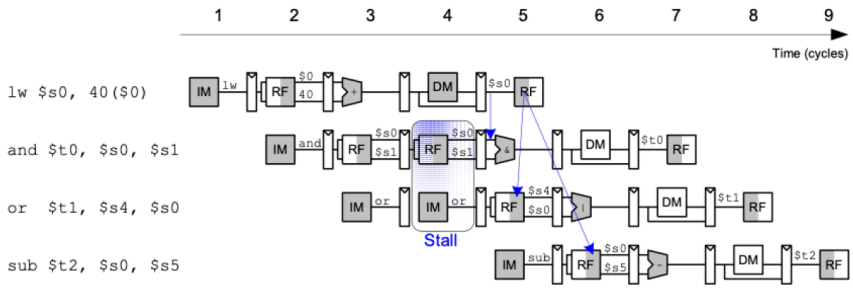


图 4: 流水线暂停示例

- 若 EX 阶段的源寄存器（rsE 或 rtE，如 $t0）匹配 MEM 阶段的写回寄存器（writeregM）且 MEM 阶段会写寄存器（regwriteM=1），forwardAE 或 forwardBE 置为 2'b10，指示数据通路将 MEM 阶段的 ALUOutM 前推至 EX 阶段 ALU 的输入 A 或 B。
- 若 rsE 或 rtE 匹配 WB 阶段的 writeregW 且 regwriteW=1（无 MEM 阶段匹配），则置为 2'b01，选择 WB 阶段的 ResultW。
- load-use 数据冒险出现在译码阶段（ID）的指令（如 add $t2, $t0, $t1）依赖 EX 阶段加载指令（如 lw $t0, 0($t3)）的写回数据（MEM 阶段的 ReadData）时。由于 lw 数据需等到 MEM 阶段，stallF、stallD 和 flushE 协作应对。当 EX 阶段为 lw 指令（memtoregE=1）且其写回寄存器（rtE，如 $t0）匹配 ID 阶段的源寄存器（rsD 或 rtD），stallF 和 stallD 置 1，分别暂停取指阶段的 PC 更新和 IF/ID 寄存器，保持 ID 阶段指令，防止新指令进入。同时，flushE 置 1，清空 EX/MEM 寄存器，插入气泡，移除 EX 阶段的无效指令。

2. 控制冒险：
   - 分支数据冒险发生在译码阶段的分支指令（如 beq $t0, $t1, label）需要寄存器 $t0 或 $t1，而这些值由 EX 阶段指令（如 add $t0, $t2, $t3）生成时。forwardAD 和 forwardBD 负责前推数据。若 ID 阶段的源寄存器（rsD 或 rtD，如 $t0）匹配 EX 阶段的写回寄存器（writeregE）且 regwriteE=1，forwardAD 或 forwardBD 置 1，指示数据通路将 EX 阶段的 ALUOutE 前推至 ID 阶段分支比较的输入（rd1D 或 rd2D），支持提前判断分支。
   - 分支依赖冒险出现在分支指令依赖 EX 或 MEM 阶段的写回数据，且数据不可前推时。例如，ID 阶段 beq $t0, $t1, label 依赖 MEM 阶段 lw $t0, 0($t4) 的 ReadData。stallF、stallD 和 flushE 再次协作。当 branchD=1（分支指令）且 EX 阶段指令会写寄存器（regwriteE=1，writeregE 匹配 rsD 或 rtD）或 MEM 阶段为 lw（memtoregM=1，writeregM 匹配 rsD 或

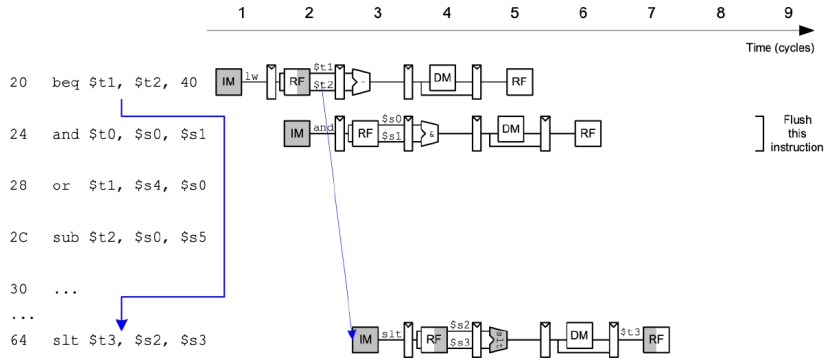


图 11: 提前判断分支

图 5: 提前判断分支示例

rtD)，stallF 和 stallD 置 1，暂停 PC 和 IF/ID 寄存器，保持分支指令；flushE 置 1，清空 EX/MEM 寄存器，移除无效指令。

2.3 数据通路模块(Datapath)

2.3.1 功能描述

数据通路模块实现五级流水线的指令执行，包括指令存储器、寄存器堆、ALU、数据存储器和流水线寄存器(IF/ID、ID/EX、EX/MEM、MEM/WB)。它处理数据传递和运算，与冒险模块协作实现前推、暂停、清空，与控制器执行指令操作。分支判断、状态转换和流水线寄存器更新在此实现。

2.3.2 接口定义

见表 2。

2.3.3 逻辑控制方法

datapath 模块是五级流水线 CPU 的核心，负责指令在取指 (IF)、译码 (ID)、执行 (EX)、访存 (MEM) 和写回 (WB) 五个阶段的处理和流动。提供寄存器地址和数据给 hazard 模块，用于冒险检测。

datapath 的逻辑控制方法通过一组控制信号和流水线寄存器，协调数据流和操作，确保指令高效且正确地执行，同时处理数据冒险和控制冒险。

流水线阶段分析:

1. IF 阶段(取指): 功能：从内存中取出指令并更新程序计数器(PC)。
   - PC 模块：PC 模块根据输入的 pc_next_jump 更新 PC 值，通常在时钟上升沿生效。
   - 默认情况下，PC 增加 4(pc_plus_4)，指向下一条顺序指令的地址。

表 2: 数据通路模块接口定义

信号名	方向	位宽	功能描述
clka	Input	1-bit	时钟信号,驱动寄存器更新。
rst	Input	1-bit	复位信号,初始化寄存器和 PC。
instr	Input	32-bit	指令存储器输入的指令。
mem_rdata	Input	32-bit	数据存储器读取的数据,用于 load 指令。
memtoreg	Input	1-bit	选择写回数据(0: ALU,1: 内存)。
alusrc	Input	1-bit	选择 ALU 第二输入(0: 寄存器,1: 立即数)。
regdst	Input	1-bit	选择写回寄存器地址(0: rt,1: rd)。
regwrite	Input	1-bit	寄存器写使能信号。
jump	Input	1-bit	跳转指令标志,控制 PC 更新。
branch	Input	1-bit	分支指令标志,控制分支跳转。
alucontrol	Input	3-bit	ALU 操作码(如加、减、与)。
regwriteE	Input	1-bit	执行阶段寄存器写使能。
regwriteM	Input	1-bit	访存阶段寄存器写使能。
memtoregE	Input	1-bit	执行阶段内存到寄存器选择。
memtoregM	Input	1-bit	访存阶段内存到寄存器选择。
pc	Output	32-bit	程序计数器地址。
alu_resultM	Output	32-bit	访存阶段 ALU 结果,用于 store 或写回。
writedataM	Output	32-bit	要写入数据存储的数据。
instrD_to_controller	Input	32-bit	译码阶段指令,传给控制器。
valid	Output	1-bit	若暂停或异常,置 0;正常置 1。

- PC 如果发生分支(pcsrc = 1),PC 被设置为分支目标地址(pc_branch)。
- 如果发生跳转(jump = 1),PC 被设置为跳转地址(jump_addra)。
- 跳转地址计算: 将指令中的 26 位偏移量左移 2 位, 与 pc_plus_4[31:28] 拼接生成 jump_addra。
- 控制信号:pc_en:控制 PC 是否更新,定义为 ~(stallF & pcsrc),即当 IF 阶段暂停且分支发生时阻止更新。jump_flush:当 jump 为 1 时清空 IF/ID 寄存器,避免执行错误的指令。

2. ID 阶段(译码): 功能:解析指令,读取寄存器数据,计算分支地址并判断是否分支。
   - IF/ID 流水线寄存器将取指阶段的指令(instr)和 pc_plus_4 传递到 ID 阶段,分别存储为 instrD 和 pc_plus_4D。
   - 寄存器堆:regfile 根据 instrD[25:21] 和 instrD[20:16] 读取两个寄存器值 rd1D 和 rd2D。
   - 立即数处理: 指令中的 16 位立即数(instrD[15:0])通过有符号扩展(sign_extend)转换为 32 位,生成 imm_extend。对于分支指令,立即数左移 2 位(imm_sl2),用于计算分支目标地址。
   - 分支地址计算: 分支目标地址 pc_branch 通过将 pc_plus_4D - 4(还原 PC)与 imm_sl2 相加计算。
   - 比较两个寄存器值 (rd1D 和 rd2D) 是否相等, 生成 equalD 信号。如果指令是分支指令 (branch = 1) 且 equalD = 1, 则 pcsrc = 1, 触发分支跳转。数据转发 (forwordAD 和 forwordBD)用于解决数据冒险,确保比较的寄存器值是最新的。
   - F_D_en: 控制 IF/ID 流水线寄存器是否更新。当 ID 阶段暂停(stallD = 1)且分支发生(pcsrc = 1)时,阻止更新。jump_flush: 清空 IF/ID 寄存器,避免跳转指令后的错误指令进入流水线。
3. EX 阶段(执行): 功能:执行 ALU 操作,选择目标寄存器地址。关键组件:
   - ID/EX 流水线寄存器将 ID 阶段的数据(如 rd1D、rd2D、imm_extend 等)传递到 EX 阶段,存储为 rd1E、rd2E、imm_extendE 等。
   - 多路选择器 (mux3 srcA_sel3 和 mux3 srcB_sel3) 根据转发信号 (forwordAE 和 forwordBE)选择 ALU 的输入:默认使用 rd1E 和 rd2E。如果有数据冒险,可能从 MEM 阶段(alu_resultM)或 WB 阶段(wd3)转发数据。
   - mux_alu_srcb 根据 alusrc 信号选择 ALU 的第二个输入是 imm_extendE(立即数)还是寄存器数据。
   - ALU 根据 alucontrol 信号执行指定运算(如加、减、与、或等),输出结果 alu_result 和零标志 zero。
   - 目标寄存器选择:多路选择器(mux_wa3)根据 regdst 信号选择目标寄存器地址 wa3:如果是 R 型指令,选择 rdE(指令的 rd 字段)。如果是 I 型指令(如 lw),选择 rtE(指令的 rt 字段)。
   - 控制信号: D_E_en: 控制 ID/EX 流水线寄存器是否清空。当分支发生 (flushE = 1 且 pcsrc = 1)时,清空寄存器以避免执行错误指令。
4. MEM 阶段(访存): 功能:传递 ALU 结果和写数据,准备内存操作。关键组件:
   - EX/MEM 流水线寄存器将 EX 阶段的数据(如 alu_result、mux3_B_result、wa3 等)传递到 MEM 阶段,存储为 alu_resultM、writedataM、wa3M 等。

- alu_resultM 通常作为内存操作的地址(例如加载或存储指令的地址)。writedataM 是要写入内存的数据(对于存储指令)。

5. WB 阶段(写回): 功能:将结果写回寄存器堆。关键组件:
   - MEM/WB 流水线寄存器将 MEM 阶段的数据(如 alu_resultM、mem_rdata、wa3M)传递到 WB 阶段,存储为 alu_resultW、mem_rdataW、wa3W。
   - 多路选择器(mux_wd3)根据 memtoreg 信号选择写回数据 wd3:如果是加载指令(memtoreg = 1),选择内存读取数据 mem_rdataW。否则,选择 ALU 结果 alu_resultW。

2.4 控制器模块(Controller)

2.4.1 功能描述

控制器根据指令操作码 (op, [31:26]) 和功能码 (funct, [5:0]) 生成控制信号 (如 memtoreg、regwrite、branch),驱动数据通路。main decoder 和 alu decoder 生成信号,通过流水线寄存器传递至 EX、MEM、WB 阶段。

2.4.2 接口定义

见表 3。

2.4.3 逻辑控制方法

2.4.4 指令解码:理解指令的意图

控制器的首要任务是分析指令,确定 CPU 需要执行的操作。指令包含两部分关键信息:**操作码**(高位部分,表明指令类型,如加法、加载、分支等)和 **功能码**(R 型指令的低位部分,指定具体运算,如加或减)。解码过程通过两个子模块完成:

- **主解码器**:读取操作码,识别指令类型(R 型、I 型、J 型等),生成一组通用控制信号。这些信号决定 CPU 在每个阶段的行为,例如是否跳转、是否写内存、是否写回寄存器等。生成的信号包括:
  - 跳转控制:是否需要跳转到新地址。
  - 分支控制:是否可能发生分支跳转。
  - ALU 输入选择:ALU 的输入是寄存器数据还是立即数。
  - 目标寄存器选择:结果写入哪个寄存器(R 型指令用 rd,I 型指令如加载用 rt)。
  - 内存操作:是否写入内存或启用内存操作。
  - 写回选择:写回寄存器的数据来自 ALU 还是内存。
  - 寄存器写使能:是否将结果写回寄存器。

  此外,主解码器生成一个中间信号,指示 ALU 需要执行的运算类型,供后续 ALU 解码器使

表 3: 控制器模块接口定义

信号名	方向	位宽	功能描述
clka	Input	1-bit	时钟信号,驱动寄存器更新。
rst	Input	1-bit	复位信号,初始化信号。
instr	Input	32-bit	译码阶段指令,生成控制信号。
jump	Output	1-bit	跳转指令标志,控制 PC 跳转。
branch	Output	1-bit	分支指令标志,触发分支比较。
alusrc	Output	1-bit	ALU 第二输入选择(0: 寄存器,1: 立即数)。
memwrite	Output	1-bit	数据存储器写使能(store)。
memtoreg	Output	1-bit	写回数据选择(0: ALU,1: 内存)。
regwrite	Output	1-bit	寄存器写使能。
regdst	Output	1-bit	写回寄存器地址选择(0: rt,1: rd)。
data_ram_ena	Output	1-bit	数据存储器读/写使能。
regwriteE	Output	1-bit	执行阶段写使能。
regwriteM	Output	1-bit	访存阶段写使能。
memtoregE	Output	1-bit	执行阶段内存到寄存器选择。
memtoregM	Output	1-bit	访存阶段内存到寄存器选择。
alucontrol	Output	3-bit	ALU 操作码(如加、减、与)。
valid	Output	1-bit	暂停或异常置 0,正常置 1。

用。

- **ALU 解码器**：针对需要 ALU 运算的指令（如加法、减法），结合主解码器的中间信号和指令的功能码，生成精确的 ALU 控制信号，指定 ALU 执行的具体运算（例如加、减、与、或等）。

通过这两个解码器，控制器将指令翻译成一组控制信号，为后续阶段提供明确的“指令”。

2.4.5 控制信号的分配与流水线传递

CPU 的五级流水线类似一条生产线，每个阶段有特定任务。控制器确保每个阶段收到正确的控制信号，与指令的数据部分保持同步。由于指令在流水线中需要多个周期完成，控制信号必须跟随指令从一个阶段传递到下一个阶段，类似接力赛中的接力棒。

- **流水线寄存器的作用**：流水线寄存器是阶段之间的“中转站”，在每个时钟周期同步存储和传递控制信号，确保信号与数据通路中的数据（如寄存器值、ALU 结果）对齐。
- **信号传递的阶段划分**：
  - **ID 阶段（译码）**：跳转和分支信号直接生效，因为它们影响程序计数器 (PC) 的更新或分支判断，需要立即作用。例如，跳转信号触发 PC 跳转到新地址，分支信号用于检查是否需要分支跳转。
  - **EX 阶段（执行）**：需要控制 ALU 和目标寄存器的信号，包括目标寄存器选择（决定结果写入哪个寄存器）、ALU 输入选择（决定 ALU 使用寄存器数据还是立即数）、ALU 控制（指定运算类型）。另外，寄存器写使能和写回数据选择信号也传递到此阶段，供冒险检测使用。
  - **MEM 阶段（访存）**：需要控制内存操作的信号，包括内存写使能（决定是否将数据写入内存）、内存操作使能（启用内存读取或写入）。寄存器写使能和写回数据选择信号继续传递，供冒险检测和后续写回使用。
  - **WB 阶段（写回）**：需要控制写回操作的信号，包括寄存器写使能（决定是否写回寄存器）和写回数据选择（决定写回的是 ALU 结果还是内存数据）。
- **传递过程**：
  1. 在 ID 阶段，主解码器和 ALU 解码器生成初始控制信号。
  2. 与 EX 阶段相关的信号存储到 ID/EX 流水线寄存器，传递到 EX 阶段。
  3. EX 阶段使用后的信号中，与内存操作相关的部分存储到 EX/MEM 流水线寄存器，传递到 MEM 阶段。
  4. MEM 阶段使用后的信号中，与写回相关的部分存储到 MEM/WB 流水线寄存器，传递到 WB 阶段。
  5. 每个寄存器在时钟信号驱动下更新，确保信号与指令的数据部分同步前进。

3 实验过程记录

3.1 存在的问题

实验过程中遇到以下主要问题：

- **控制信号生成错误**：部分 I 型指令（如 addi）的 alusrc 信号未正确设置为 1，导致 ALU 错误使用寄存器数据而非立即数。
- **分支指令延迟**：分支指令（beq）的分支信号（branch）在 ID 阶段生成后，未及时清空后续流水线，导致错误指令进入 EX 阶段。
- **J 型指令跳转延迟**：跳转指令（j）的 jump 信号未及时生效，造成 PC 更新延迟。

3.2 解决方案

针对上述问题，采取了以下解决方案：

- **修复信号生成**：检查主解码器逻辑，修正 addi 等 I 型指令的 alusrc 信号生成规则，确保选择立即数。
- **解决分支指令延迟**：在控制器中增强与数据通路的协作，通过 jump 和 branch 信号直接触发流水线清空机制。引入 flush 信号，在分支发生时清空 EX 阶段的流水线寄存器，避免错误指令执行。
- **改进跳转处理**：优化 jump 信号的触发逻辑，确保 PC 在 ID 阶段立即更新到跳转地址。

4 实验结果及分析

仿真图如图 6 所示。控制台输出如图 7 所示。仿真图中：包含 pc_now（取指阶段 pc 的值）、instr（处于译码阶段的指令）、aluresult（执行阶段计算出来 alu 的结果）、RD1(执行阶段的 alu 的第一个输入，第一个寄存器的值）、RD2（执行阶段的 alu 的第二个输入，立即数或者第二个寄存器的值）、wd3（写回阶段写回寄存器的值）等信号以及执行阶段的 rs、rt、rd。

multirow listings xcolor

A Datapath 代码

```
module datapath(
    input wire clka,rst,
    input wire [31:0] instr,
    input wire [31:0] mem_rdata,     //data_ram中读出的数据
    output wire [31:0] pc,
    output wire [31:0] alu_resultM,
    output wire [31:0] writedataM,
    input wire memtoreg,
```

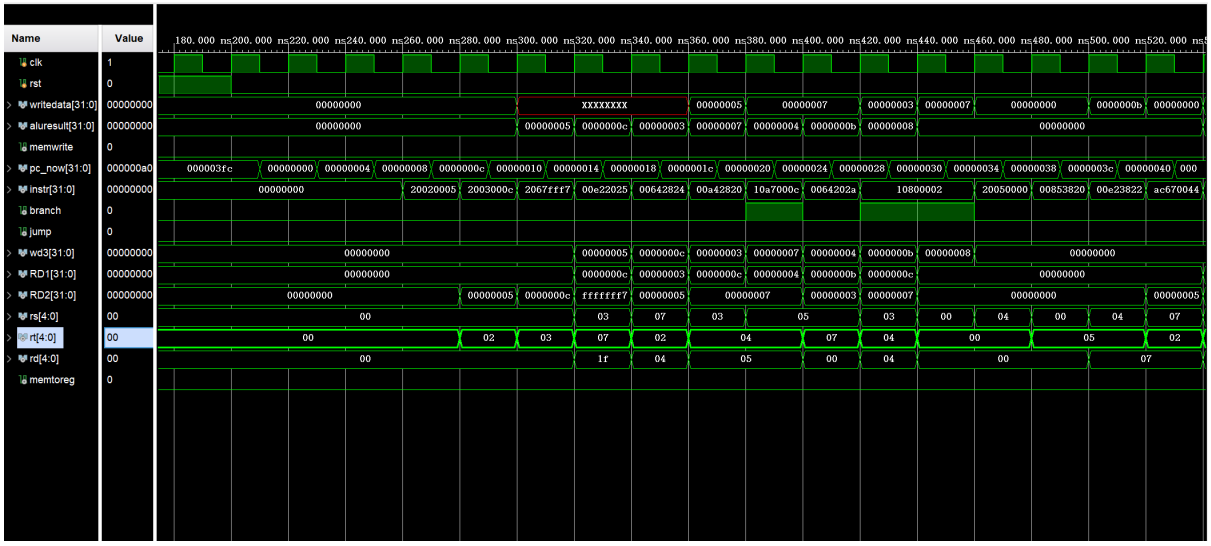


图 6: 仿真图

```
Vivado Simulator 2019.2
Time resolution is 1 ps
 Block Memory Generator module loading initial data...
 Block Memory Generator data initialization complete.
Block Memory Generator module testbench.dut.imem.inst.native_mem_mapped_module.blk_mem_gen_v8_4_4_inst is using a behavioral model for simulation which will not precisely model memory col
Block Memory Generator module testbench.dut.dmem.inst.native_mem_mapped_module.blk_mem_gen_v8_4_4_inst is using a behavioral model for simulation which will not precisely model memory col
Simulation succeeded
$stop called at time : 630 ns : File "E:/vivado_project/wrap_lab4/wrap_lab4/rtl/sim/testbench.v" Line 31
relaunch_sim: Time (s): cpu = 00:00:01 ; elapsed = 00:00:06 . Memory (MB): peak = 1330.496 ; gain = 0.000
```

图 7: 控制台输出

```
9     input wire alusrc,
10    input wire regdst,
11    input wire regwrite,
12    input wire jump,
13    input wire branch,
14    input wire regwriteM,
15    input wire memtoregE,
16    input wire regwriteE,
17    input wire memtoregM,
18    input wire [2:0] alucontrol,
19    output wire [31:0]
          instrD_to_controller//从datapath中传出，由于instr必须为D阶段的才能使controller与
20    );
21    wire [31:0] pc_next;          //pc+4后的下一位pc
22    wire [31:0] pc_next_jump;
          //选择pc+4?branch后，再次选择是否jump后的PC值
23    wire [31:0] rd1D;             //regfile输出的rd1
24    wire [31:0] rd2D;             //regfile输出的rd2
25    wire [31:0] imm_extend;       //i型指令16位imm有符号扩展成32位
26    wire [31:0] alu_result;       //alu计算结果
27    wire [31:0] alu_srcB;         //alusec控制得到的alu_srcB
28    wire [31:0] wd3;              //写regfile数据(ReadData ? ALUOut)
29    wire [4:0] wa3;               //写regfile的寄存器号（rt ? rd）
30    wire [31:0] imm_sl2;          //imm_extend左移2位（在branch指令下工作）
```

```
31      wire [31:0] pc_branch;          //branch分支地址
32      wire [31:0] pc_plus_4;          //pc+4
33      wire [31:0] instr_jump_offset_sl2;
            //jump指令中低26位偏移地址左移两位后的结果（这里是32位，高四位后面舍弃）
34      wire pcsrc;                     //判断pc 在branch指令下能否执行
35      wire zero;                      //提前判断分支，比较branch指令中是否相等
36      wire [31:0] mux3_A_result;
37      wire [31:0] mux3_B_result;
38      //F-D间信号
39      wire [31:0] instrD;
40      wire [31:0] pc_plus_4D;
41      //D-E间信号
42      wire [31:0] rd1E;
43      wire [31:0] rd2E;
44      wire [31:0] pc_plus_4E;
45      wire [31:0] imm_extendE;
46      wire [4:0] rsE;                 //instr[25:21]，用于hazard模块
47      wire [4:0] rtE;                 //filereg回写地址时 rt的地址 传入mux wa3
48      wire [4:0] rdE;                 //filereg回写地址时 rd的地址 传入mux wa3
49      //E-M间信号
50      wire [31:0] pc_branchM;         //branch指令下pc跳转结果
51      wire [4:0] wa3M;                //选择rd还是rs写回数据的结果
52      //M-W间信号
53      wire [31:0] alu_resultW;        //回写的aluresult，送到选择器去
54      wire [31:0] mem_rdataW;         //Data_ram中读出，送到writeback阶段的选择器
55      wire [4:0] wa3W;                //选择rd还是rs写回数据的结果
56      wire zeroM;
57      //hazard传出的延迟与刷新信号
58      wire stallF,stallD,flushE;
59      //数据前推控制器
60      wire [1:0] forwordAE,forwordBE;
61      wire forwordAD,forwordBD;
62      wire equalD;
63      //pcSrc的判断
64      assign pcsrc = branch & equalD;
65      mux2 #(32) mux_pc_next(
66          .a(pc_branch),              //branch的跳转
67          .b(pc_plus_4),
68          .s(pcsrc),                  //连接pcSrc
69          .y(pc_next)
70          );
71      //j指令的低26位为字地址，左移两位得到字节地址，后面与PC高4位拼接
72      shift_2 sl2_pc_jump(
73          .a(instrD),
74          .y(instr_jump_offset_sl2)
```

```
            //得到jump指令中偏移地址左移两位后的结果，此处为32位，高4位舍弃
        );
    //j指令的跳转地址
    wire [31:0] jump_addra;
    assign jump_addra = {pc_plus_4[31:28],instr_jump_offset_sl2[27:0]} +4;
    //PC跳转
    mux2 #(32) mux_pc_jump(
        .a(jump_addra), //jump指令下的地址跳转
        .b(pc_next),    //PC+4
        .s(jump),       //jump判断信号
        .y(pc_next_jump)
        );
    //判断在branch指令时是否要清空pc（流水线延迟导致branch下一条指令执行）
    wire pc_en;
    assign pc_en = ~(stallF & pcsrc);
    // 新增信号：Jump 清空信号
    wire jump_flush = jump;  // Jump 指令在 D 阶段时置 1

    //PC
    pc pc_module(.clk(clka),.rst(rst),.en(1'b1),.din(pc_next_jump),.q(pc));
    //PC+4
    adder pc_plus_4_module(.a(pc),.b(32'h4),.y(pc_plus_4));
//流水线寄存器控制信号
    wire F_D_en;
    assign F_D_en = ~(stallD & pcsrc);
    //F-D数据传输
    flopenrc #(32)
        r1D(.clk(clka),.rst(rst),.en(F_D_en),.clear(jump_flush),.d(instr),.q(instrD));
    flopenrc #(32)
        r2D(.clk(clka),.rst(rst),.en(F_D_en),.clear(jump_flush),.d(pc_plus_4),.q(pc_plus_

    assign instrD_to_controller =
        instrD;//从datapath中传出，由于instr必须为D阶段的才能使controller与其匹配。同时in

    //instr低16位偏移地址扩展与左移
    sign_extend sign_extend(.a(instrD[15:0]),.y(imm_extend));
    shift_2 sl2(.a(imm_extend),.y(imm_sl2));
    //计算branch时PC已经+4了，要还原
    wire [31:0] pc_plus_4D_for_brach;
    assign pc_plus_4D_for_brach = pc_plus_4D - 4;
    adder
        pc_branch_module(.a(pc_plus_4D_for_brach),.b(imm_sl2),.y(pc_branch));
    //寄存器堆
    regfile regfile(
        .clk(clka),
```

```
115         .we3(regwrite),                 //写使能
116         .ra1(instrD[25:21]),            //读寄存器号1
117         .ra2(instrD[20:16]),            //读寄存器号2
118         .wa3(wa3W),                     //写地址
119         .wd3(wd3),                      //写数据
120         .rd1(rd1D),                     //读数据1
121         .rd2(rd2D)                      //读数据2
122         );
123
124     wire [31:0] rd1_1sle_branch_A,rd2_1sle_branch_B;
125     //rd1_decode_sle_branch_A
126     mux2 #(32) mux_rd1_1sle_branch_A(
127         .a(alu_result),                 //jump指令下的地址跳转
128         .b(rd1D),
129         .s(forwordAD),                  //jump
130         .y(rd1_1sle_branch_A)
131         );
132     //rd2_decode_sle_branch_B
133     mux2 #(32)
        mux_rd2_1sle_branch_B(.a(alu_result),.b(rd2D),.s(forwordBD),.y(rd2_1sle_branch_B
134     //提前判断分支减少控制冒险
135     assign equalD = (rd1_1sle_branch_A == rd2_1sle_branch_B);
136     //判断branch时是否要清空excute级流水线
137     wire D_E_en;
138     assign D_E_en = flushE & pcsrc;
139
140     //D-E数据传输
141     flopenrc #(32)
        r1E(.clk(clka),.rst(rst),.en(1'b1),.clear(D_E_en),.d(rd1D),.q(rd1E));
142     flopenrc #(32)
        r2E(.clk(clka),.rst(rst),.en(1'b1),.clear(D_E_en),.d(rd2D),.q(rd2E));
143     flopenrc #(5)
        r3E(.clk(clka),.rst(rst),.en(1'b1),.clear(D_E_en),.d(instrD[20:16]),.q(rtE));
144     flopenrc #(5)
        r4E(.clk(clka),.rst(rst),.en(1'b1),.clear(D_E_en),.d(instrD[15:11]),.q(rdE));
145     flopenrc #(32)
        r5E(.clk(clka),.rst(rst),.en(1'b1),.clear(D_E_en),.d(pc_plus_4D),.q(pc_plus_4E))
146     flopenrc #(32)
        r6E(.clk(clka),.rst(rst),.en(1'b1),.clear(D_E_en),.d(imm_extend),.q(imm_extendE)
147     flopenrc #(5)
        r7E(.clk(clka),.rst(rst),.en(1'b1),.clear(D_E_en),.d(instrD[25:21]),.q(rsE));
148
149     //连接regfile的wa3,选择写入结果的地址是rt (lw) 还是rd (r-type)
150     mux2 #(5) mux_wa3(
151         .a(rdE),                //rt的地址
```

```
152             .b(rtE),                //rd的地址
153             .s(regdst),             //regdst
154             .y(wa3)
155             );
156 
157     mux3 #(32)
            srcA_sel3(.d0(rd1E),.d1(wd3),.d2(alu_resultM),.s(forwordAE),.y(mux3_A_result));
158 
159     mux3 #(32)
            srcB_sel3(.d0(rd2E),.d1(wd3),.d2(alu_resultM),.s(forwordBE),.y(mux3_B_result));
160 
161     //alu_srcB
162     mux2 #(32)
            mux_alu_srcb(.a(imm_extendE),.b(mux3_B_result),.s(alusrc),.y(alu_srcB));
163     //ALU
164     alu
            alu(.a(mux3_A_result),.b(alu_srcB),.op(alucontrol),.result(alu_result),.zero(zero
165 
166     //E-M数据传输
167     flopenrc #(32)
            r1M(.clk(clka),.rst(rst),.en(1'b1),.clear(1'b0),.d(alu_result),.q(alu_resultM));
168     flopenrc #(1)
            r2M(.clk(clka),.rst(rst),.en(1'b1),.clear(1'b0),.d(zero),.q(zeroM));
169     flopenrc #(32)
            r3M(.clk(clka),.rst(rst),.en(1'b1),.clear(1'b0),.d(mux3_B_result),.q(writedataM)
170     flopenrc #(32)
            r4M(.clk(clka),.rst(rst),.en(1'b1),.clear(1'b0),.d(pc_branch),.q(pc_branchM));
171     flopenrc #(5)
            r5M(.clk(clka),.rst(rst),.en(1'b1),.clear(1'b0),.d(wa3),.q(wa3M));
172 
173     //M-W数据传输
174     flopenrc #(32)
            r1W(.clk(clka),.rst(rst),.en(1'b1),.clear(1'b0),.d(alu_resultM),.q(alu_resultW))
175     flopenrc #(32)
            r2W(.clk(clka),.rst(rst),.en(1'b1),.clear(1'b0),.d(mem_rdata),.q(mem_rdataW));
176     flopenrc #(5)
            r3W(.clk(clka),.rst(rst),.en(1'b1),.clear(1'b0),.d(wa3M),.q(wa3W));
177 
178     //wd3
179     mux2 #(32)
            mux_wd3(.a(mem_rdataW),.b(alu_resultW),.s(memtoreg),.y(wd3));
180 
181     hazard
            hazard(.rst(rst),.rsD(instrD[25:21]),.rtD(instrD[20:16]),.rsE(rsE),.rtE(rtE),
182          .regwriteE(regwriteE),.regwriteM(regwriteM),.regwriteW(regwrite),.memtoregE(mem
```

```
183         .memtoregM(memtoregM),.branchD(branch),.writeregE(wa3),.writeregM(wa3M),.writereg
184         .forwordAE(forwordAE),.forwordBE(forwordBE),.forwordAD(forwordAD),.forwordBD(for
185         .stallF(stallF),.stallD(stallD),.flushE(flushE));
186 endmodule
```

B Hazard 代码

```
1  module hazard(
2      input wire rst,
3      input wire [4:0] rsD,
          //instr2[25:21] (同一时刻rsD和rsE对应先后两条指令的rs字段, 并不相同)
4      input wire [4:0] rtD,          //instr2[20:15]
5      input wire [4:0] rsE,          //instr1[25:21]
6      input wire [4:0] rtE,          //instr1[20:15]
7      input wire regwriteE,
          //寄存器堆的写使能信号 (E、M、W表示excute、memory和writeback阶段)
8      input wire regwriteM,
9      input wire regwriteW,
10     input wire memtoregE,
          //判断写回寄存器堆的数据是sw的ReadData(0)还是R的ALUOut(1)
11     input wire memtoregM,
12     input wire branchD,            //提前判断分支
13     input wire [4:0] writeregE,  //寄存器堆的写地址, 连接wa3W
14     input wire [4:0] writeregM,
15     input wire [4:0] writeregW,
16     output [1:0] forwordAE,        //在excute阶段控制mux3选择SrcA (数据冒险)
17     output [1:0] forwordBE,        //在excute阶段控制mux3选择SrcB
18     output [1:0] forwordAD,        //在decode阶段控制二选一选择regfile
          rd1出来的数据 (控制冒险下的数据冒险)
19     output [1:0] forwordBD,        //在decode阶段控制二选一选择regfile
          rd2出来的数据
20     output reg stallF,             //instr fetch级暂停
21     output reg stallD,             //decoder暂停
22     output reg flushE              //excute读到的流水线需要清空
23     );
24 //数据前推解决R指令和前两条lw指令的数据冒险
25     /*ALU端口SrcAE的数据可能来自: (注意判断reE!=0, 否则读保留寄存器直接输出)
26          寄存器堆 (无冒险情况下): forwordAE=00、SrcAE=RsD
27          数据存储器(lw的数据冒险): forwordAE=01、SrcAE=ResultW (lw指令写回寄存器堆在MEM阶
28          ALUOut (ALU运算的数据冒险): forwordAE=10、SrcAE=ALUOutM (R型指令写回寄存器堆在W
29     */
30     assign forwordAE = ((rsE != 5'b0) & (rsE == writeregM) & regwriteM) ?
          2'b10:      //前一条指令是R型, 直接将ALUOut传回
```

```
31                          ((rsE != 5'b0) & (rsE == writeregW) & regwriteW) ?
                                2'b01:      //前两条指令是lw
32                           2'b00;
33      assign forwordBE = ((rtE != 5'b0) & (rtE == writeregM) & regwriteM) ?
            2'b10:      //SrcBE同SrcAE
34                          ((rtE != 5'b0) & (rtE == writeregW) & regwriteW) ?
                                2'b01:
35                           2'b00;
36  //提前判断分支解决beq后2、3条指令的控制冒险时出现的数据冒险
37      assign forwordAD = ((rsD != 5'b0) & (rsD == writeregE) & regwriteE);
                        //前一条指令要写回寄存器堆且该数据被beq指令所用
38      assign forwordBD = ((rtD != 5'b0) & (rtD == writeregE) & regwriteE);
39  //流水线暂停解决lw后一条指令需要用寄存器堆数据带来的数据冒险、beq需要用前一条指令写回寄
40      /*lw指令写入寄存器堆的地址为rt，因此下一条指令若要用到rt则需要暂停，即rsD==rtE或rtD=
41        beq指令需要rs、rt号寄存器的数据，若上一条指令要写到该寄存器，则需要暂停流水线
42      */
43      wire lwstall,branch_stall;
            //流水线暂停信号
44      assign lwstall = (((rsD == rtE) | (rtD == rtE)) & memtoregE);
            //判断前一条指令需要对寄存器堆写入(memturegE)并且写入地址rtE与被当前指令用到
45      assign branch_stall=(
            branchD&regwriteE&((writeregE==rsD)|(writeregE==rtD)) )
            //当前指令为branch、上一条指令要写寄存器堆且写的数据当前要用
46                          |(
                                branchD&memtoregM&((writeregM==rsD)|(writeregM==rtD))
                                );
                                //当前指令为branch、上2条指令要写寄存器堆且写的数据当前要用
47      always @(*) begin
48          stallF = rst? 1'b0 : (lwstall | branch_stall);
                //若被重置则全部清零
49          stallD = rst? 1'b0 : (lwstall | branch_stall);
                //lw带来的数据冒险或提前判断分支带来的数据冒险都可以暂停流水线
50          flushE = rst? 1'b0 : (lwstall | branch_stall);
                //lw/beq下一条已经执行的需要清空
51      end
52
53  endmodule
```

C Controller 代码

```
1   module controller(
2       input clka, rst,
3       input wire [31:0] instr,
```

```
4       output wire jump, branch, alusrc, memwrite, memetoreg, regwrite,
            regdst, data_ram_ena, regwriteE, memtoregM,
5       output wire regwriteM,
6       output wire memtoregE,
7       output wire [2:0] alucontrol
8   );
9   //根据instr[31:26]和instr[5:0]解码
10      wire [1:0] aluop;       //Main Decode输出的aluop信号, 传入ALU Decoder
11      wire [7:0] sigsD;       //Main Decode输出的8bit控制信号
12      //main_dec 实例化
13      main_dec Main_Decoder(.op(instr[31:26]), .sigs(sigsD), .aluop(aluop));
14      wire [2:0] alucontrolD; //ALU
            Decoder输出的ALU控制信号, 传入流水线寄存器
15      //alu_dec 实例化
16      alu_dec ALU_Control(.funct(instr[5:0]), .op(aluop),
            .alucontrol(alucontrolD));
17      assign jump = sigsD[7];
            //jump和branch信号不用继续传输, 直接传给下一条指令以减少控制冒险
18      assign branch = sigsD[3];
19
20  //流水线寄存器DE间的数据进出: {regwrite,regdst,alusrc,memwrite,memetoreg,data_ram_ena}和
21      wire [5:0] sigsE;
            //{regwrite,regdst,alusrc,memwrite,memetoreg,data_ram_ena}
22      wire [2:0] alucontrolE; //从流水线寄存器DE读出的ALU控制信号
23      floprc #(6) r1E(.clk(clka), .rst(rst), .clear(1'b0), .d({sigsD[6:4],
            sigsD[2:0]}), .q(sigsE));
24      floprc #(3) r2E(.clk(clka), .rst(rst), .clear(1'b0), .d(alucontrolD),
            .q(alucontrolE));
25      assign regdst = sigsE[4];
26      assign alusrc = sigsE[3];
27      assign alucontrol = alucontrolE;
28      assign memtoregE = sigsE[1];
29      assign regwriteE = sigsE[5];
30
31  //流水线寄存器EM间的数据进出: {regwrite,memwrite,memetoreg,data_ram_ena}
32      wire [3:0] sigsM;
33      floprc #(4) r1M(.clk(clka), .rst(rst), .clear(1'b0), .d({sigsE[5],
            sigsE[2:0]}), .q(sigsM));
34      assign memwrite = sigsM[2];
35      assign data_ram_ena = sigsM[0];
36      assign regwriteM = sigsM[3];   //传入datapath中的hazard需要
37      assign memtoregM = sigsM[1]; //传入datapath中的hazard需要
38
39  //流水线寄存器MW间的数据进出: {regwrite,memwrite,memetoreg}
40      wire [1:0] sigsW;
```

```
41      floprc #(2) r1W(.clk(clka), .rst(rst), .clear(1'b0), .d({sigsM[3],
            sigsM[1]}), .q(sigsW));
42      assign regwrite = sigsW[1];
43      assign memetoreg = sigsW[0];
44
45  endmodule
```